

CUDA Lab 02: Advanced Geometries & Stencils

Results Report

Student ID	<fill in>
Allocated GPU Node	<fill in from nvidia-smi / Colab>
CUDA Compute Capability	<fill in>
Official Verification Token	<run verify_submission.py on your GPU>

Provenance of the numbers: Tasks 2-4 below were produced by running the submitted kernels under the Numba CUDA simulator (NUMBA_ENABLE_CUDASIM=1) on a CPU-only machine, at reduced problem sizes because the simulator is slow. They verify correctness, not GPU performance. Items marked <fill in> must come from a real run on your GPU; they are intentionally not invented.

Task 1: Warp Divergence Benchmark ($N = 2^{20}$, 1,000 iterations/element)

Kernel-only time (no H2D/D2H), 1 warm-up launch, mean of 10 trials via `cuda.synchronize()` and `time.perf_counter()`. Run `python task1_divergence.py` on the GPU and paste the values:

Kernel	Avg Time (ms)	Slowdown vs A
A (Uniform)	<measure>	1.00x
B (Interleaved, <code>idx % 2</code>)	<measure>	<measure>
C (Warp-aligned, <code>idx // 32</code>)	<measure>	<measure>

Expected behavior / analysis. A warp executes 32 threads in lockstep. In Kernel B, even and odd lanes of every warp take different branches, so the hardware runs the multiply-accumulate path with odd lanes masked, then the subtract-divide path with even lanes masked; the two paths run back to back. Kernel B therefore costs roughly the sum of both paths, while A and C cost about one path per warp. In Kernel C the branch condition is constant across each warp (`warp_id = idx // 32`), so no serialization occurs. C should land near A, with some difference because subtract-divide is slower than multiply-add. Division is more expensive than FMA, so C's odd warps cost more than A's.

Task 2: 1D Stencil with Halo Replication

TASK 2 PASSED: MAX DELTA = 0.0 (N = 10,007, simulator)

Boundary guard `if idx < N; idx 0 reads d_in[0] as its left neighbor and idx N-1 reads d_in[N-1] as its right neighbor, matching np.pad(..., mode='edge'). On your GPU, run the script for  $N = 100,007$  and paste the printed line (expect a delta at or near  $1e-8$  or below, well under  $atol = 1e-4$ ).`

Task 3: Grid-Stride Scaling

TASK 3 PASSED: all 100,000 elements == 4.25
(64 blocks x 256 threads, simulator)

Launch is fixed at $64 \times 256 = 16,384$ threads regardless of N ; each thread loops `range(cuda.grid(1), N, cuda.gridsize(1))`. For the full $N = 2^{24} = 16,777,216$ test, each thread handles 1,024 elements. Run `python task3_grid_stride.py` on the GPU for that line.

Task 4: 2D Sobel-X

TASK 4 PASSED: MAX DELTA = 0.0 (96x96 random image, simulator)
Flat-field interior `max |grad| = 0.0` | borders zero: True

Launch uses (16, 16) threads per block and a grid of $\text{ceil}(\text{cols}/16) \times \text{ceil}(\text{rows}/16)$ blocks (128 x 128 for 2048 x 2048). Border pixels are written as 0.0. Run `python task4_sobel_2d.py` on the GPU for the 2048 x 2048 result.

verify_submission.py

The integrity token hashes your Student ID, the GPU device name and the kernel outputs, so it can only be produced by running `python verify_submission.py` on your own GPU runtime. Paste it into the header above and into README.md.